

07 The Transport Layer

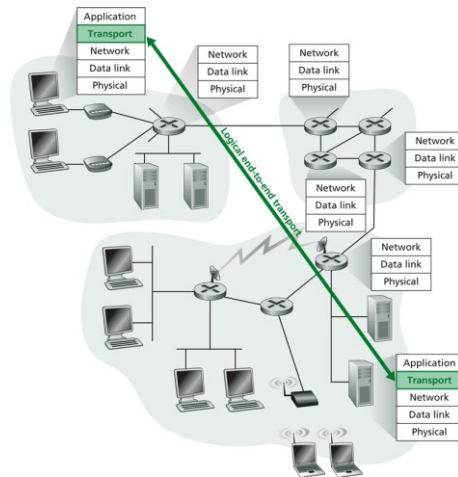
07.01-07.03 Transport services,
Transport protocol elements,
User Datagram Protocol (UDP)

The Transport Layer

- The transport layer should offer a effective, reliable, efficient service to its users
- Transport layer users are application layer processes
- Transport layer uses services provided by network layer
- The transport code is entirely implemented on users host, while network layer is mostly executed on routers

The Transport Layer

- The transport layer provides a *logical* end-to-end data communication channel between application processes in different hosts
- The underlying infrastructure is transparent to the application processes: all works as if hosts were directly connected even if they are in different places of the earth
- Hence: the transport layer provides reliable and efficient data transport from source to destination, independently from the network or from the physical nets used

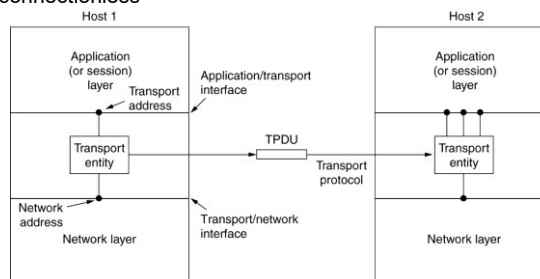


♦ The transport layer provides logical rather than physical communication between application processes.

Copyright © 2005 Pearson
Addison Wesley All rights reserved

Services Provided to the Upper Layers

- The software/hardware that implements transport layer functionalities is called *transport entity*, the exchanged information is called TDP (transport protocol data unit)
- This is done by the **transport entity** (HW or SW implemented → OS kernel, user process, packet library or NIC)
- The use of transport layer allows the transport service to be reliable even if the network layer is unreliable
- Generally services offered by transport layer are:
 - Reliable connection oriented
 - Unreliable connectionless



The network, transport, and application layers.

Transport Service Primitives

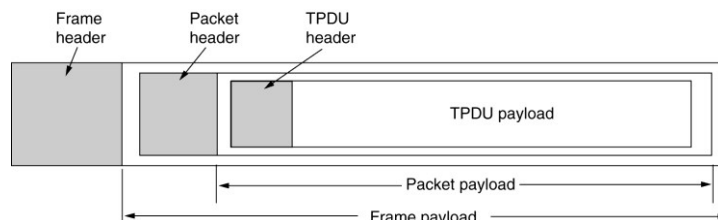
- Users should access to transport service
- This is accomplished by the interface that let the users see the **transport service primitives**
- Usually transport service primitives are API (Application Program Interfaces): function prototypes to be used by applications to access the transport services
- The primitives for a simple transport service connection oriented should allow to the application processes to establish, use and release the connections

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

The primitives for a simple (hypothetical) transport service.

Transport Service Primitives

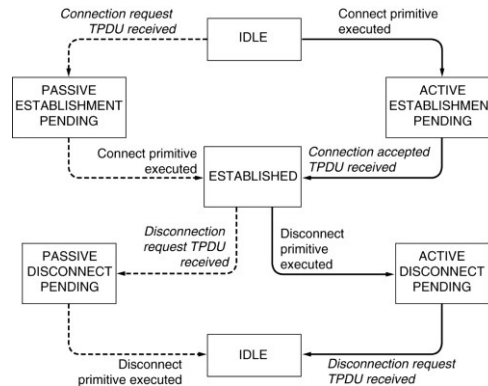
- TPDU (*Transport Protocol Data Unit*) is the name for messages sent from the transport entity to other transport entities
- TDPUs are also called segments
- Client-server example:
 - Client: CONNECT (a TDPU CONNECTION REQUEST is sent to the server)
 - Server: Transport entity checks if server is blocked on a LISTEN. Server is un-blocked and a TDPU CONNECTION ACCEPTED is sent.
 - Client: when CONNECTION ACCEPTED is received the client is un-blocked and the connection is established
 - Data can then be exchanged through SEND and RECEIVE primitives
 - Connection are released through DISCONNECT



The nesting of TPDUs, packets, and frames.

Transport Service Primitives

- A state diagram for a simple connection management scheme. Transitions labeled in *italics* are caused by packet arrivals. The solid lines show the client's state sequence. The dashed lines show the server's state sequence.



Berkeley sockets

- The first 4 are executed in this order by a server
- Once connection is established both sides can receive/send on the full duplex connection

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

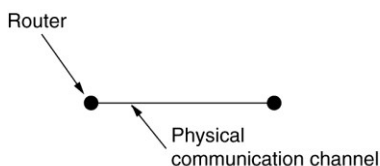
The socket primitives for TCP.

Elements of Transport Protocols

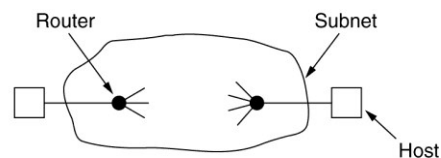
- Addressing
- Connection Establishment
- Connection Release
- Flow Control and Buffering
- Multiplexing

Transport Protocol

- Transport protocols are similar to data link protocols: both deal with error control, sequence ordering and flow control
- But: they operate in different environments
- Transport layer requires addressing
- Connection handling is more complicated
- Data can be buffered or delayed when traveling from source to destination
- Different control flow algorithms are needed to manage a dynamic number of connections



(a)

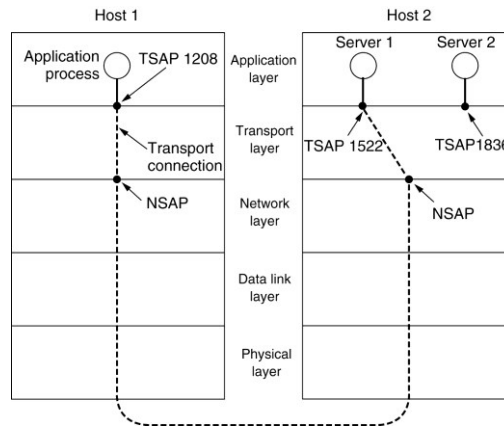


(b)

(a) Environment of the data link layer. (b) Environment of the transport layer.

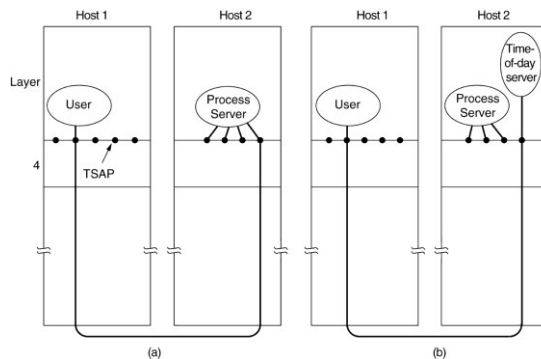
Addressing

- Transport addresses are defined to allow a process to listen for possible connection requests
- In the Internet these final points are called **ports**
- In general we can call them TSAP (Transport Service Access Point)
- NSAP (Network Service Access Point) are corresponding final points in the network layer (e.g. IP addresses)



Connection Establishment

- If stable (well known) TSAP addresses cannot be used (e.g. it is not useful to keep process servers all active) 2 alternative solutions can be used
- A protocol for initial connection: use of a process server that acts as proxy for rarely used server
- A name server (directory server) that answers with the TSAP of a given server

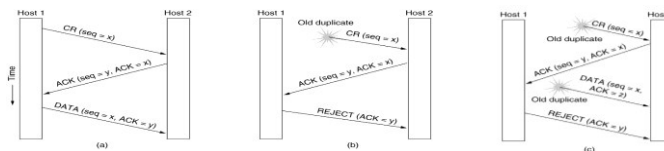


Connection Establishment

- A problem in connection establishment is the existence of possible duplicate delayed packets
- It is important to ensure that two packets of the same connection have different sequence numbers
- A solution can be to use a counter increased at regular intervals working even if the host is blocked
- The k least significant bit can be taken as starting sequence number
- The sequence space should be wide enough to guarantee that when seq.nr. re-start old TDPU with same nr.seq have been removed (e.g. TTL)

Connection Establishment

- Both receiver and sender need to agree on setting up a connection. Also this phase has to face the problem of delayed duplicated packets
- Tomlinson (1975) proposed the 3-way handshake:
 - Host 1 chooses a seq.nr. x and sends a TDPU CONNECTION REQUEST
 - Host 2 replies with a TDPU ACK that acknowledges x and proposes its seq.nr. y
 - Host 1 acknowledges y in the next TDPU (that can also carry data) confirming its seq.nr. x
- The strategy is robust to the problem of old duplicates

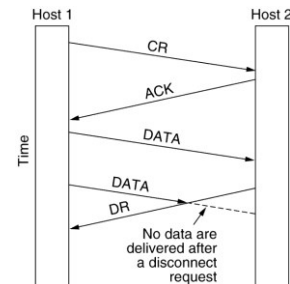


Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST.

- (a) Normal operation,
- (b) Old CONNECTION REQUEST appearing out of nowhere.
- (c) Duplicate CONNECTION REQUEST and duplicate ACK.

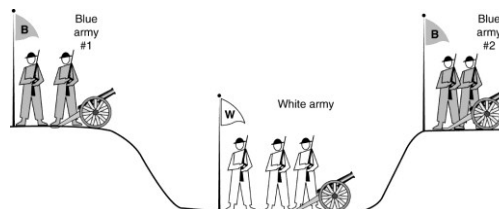
Connection Release

- Also the connection release poses some issues
- Possible releases:
 - Symmetric release
 - Asymmetric release
- Asymmetric release may cause abrupt disconnection with loss of data (see figure)



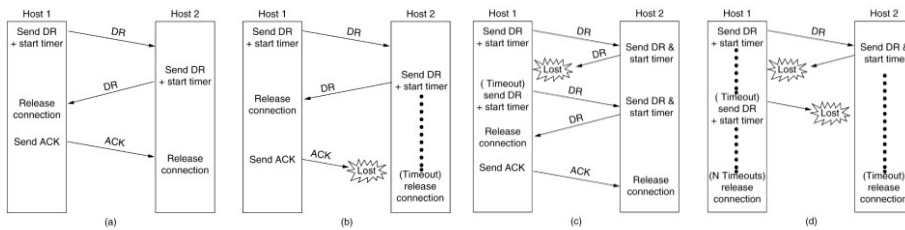
Connection Release

- Symmetric release requires both sides agree on the release, but
 - The 2 army problem (i.e. the problem of being sure that both parts agree) has no solution
 - An acceptable solution is a 3-way handshake with timeout



Connection Release

- Four protocol scenarios for releasing a connection.
 - (a) Normal case of a three-way handshake
 - (b) final ACK lost
 - (c) Response lost
 - (d) Response lost and subsequent DRs lost



Congestion Control and Buffering

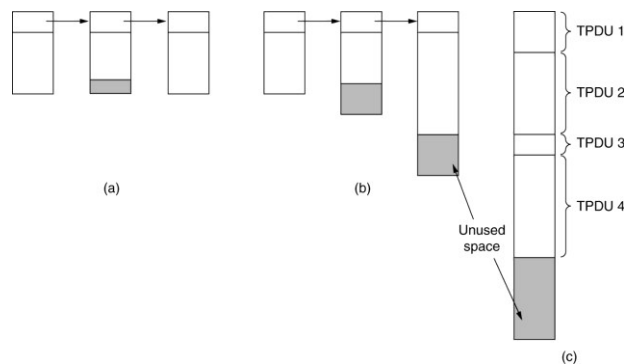
- Congestion control is also managed at network layer: both layers make use of sliding window mechanisms
- While a router has usually few lines, a host may have many connections
- Differences: practical implementations of buffering
- If the subnet provides an unreliable service:
 - Source host must store sent TDPUs (it could be needed re-transmissions)
 - Destination host keeps a given pool of buffers shared by all connections. If no buffer is available the TDPUs are discarded
- If the subnet provides a reliable service:
 - If source knows that receiver has empty buffers, it could avoid storage of TDPUs
 - It must store TDPUs if the receiver Transport is not able to allocate space from them

Flow Control and Buffering

- Since different connection may have rapidly changing features the solution can be
 - Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost TPDU.

A	Message	B	Comments
1 →	< request 8 buffers >	→	A wants 8 buffers
2 ←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3 →	<seq = 0, data = m0>	→	A has 3 buffers left now
4 →	<seq = 1, data = m1>	→	A has 2 buffers left now
5 →	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6 ←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7 →	<seq = 3, data = m3>	→	A has 1 buffer left
8 →	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9 →	<seq = 2, data = m2>	→	A times out and retransmits
10 ←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11 ←	<ack = 4, buf = 1>	←	A may now send 5
12 ←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13 →	<seq = 5, data = m5>	→	A has 1 buffer left
14 →	<seq = 6, data = m6>	→	A is now blocked again
15 ←	<ack = 6, buf = 0>	←	A is still blocked
16 ...	<ack = 6, buf = 4>	←	Potential deadlock

Tradeoffs in a reliable network



- (a) Chained fixed-size buffers. (b) Chained variable-sized buffers.
- (c) One large circular buffer per connection.

Tradeoffs in a reliable network

- Fixed size buffers – what should be the size?
- Variable size buffers – better memory utilization but complicated buffer management
- One large circular buffer per connection: good if all connections are heavily loaded but poor otherwise.

In practice, a combination might be used depending upon the application.

Tradeoffs between buffering at sender and at receiver

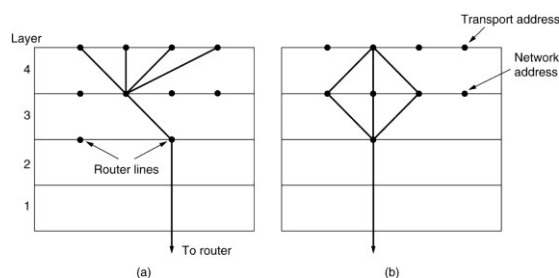
- Low bandwidth bursty application like interactive terminal,
 - having dedicated buffers is not a good idea, buffers must be acquired dynamically.
 - Hence, buffer space at the receiver is not guaranteed, therefore sender must keep.
- High bandwidth traffic like file transfer
 - Receiver must dedicate buffers to allow smooth and fast flow of data.
 - Hence, sender need not keep.

Negotiating buffer space in CR

- As connections are opened and closed, and the traffic pattern changes, sender and receiver must be able to adjust their buffer allocation dynamically.
- Transmission Protocol should allow the sender to request for buffer space at the receiver in Connection Request,
- And the receiver to inform the sender as to how much buffer space it has for the sender, say in Connection Accepted.

Multiplexing

- If the host has only 1 network address, all transport connections must use that address (upward multiplexing (a))
- If we have a single transport address that open more network connections to increase bandwidth (downward multiplexing (b))



Crash Recovery

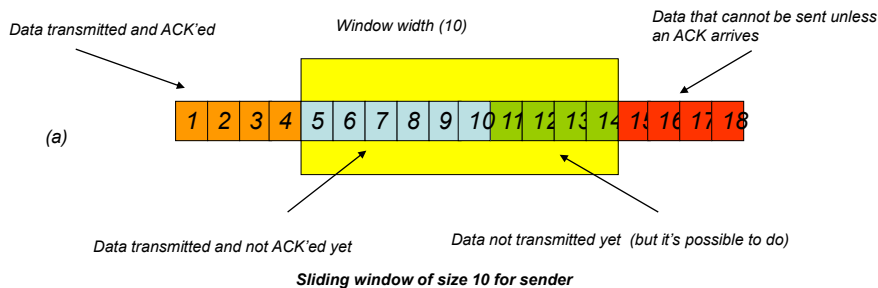
- Problem:** A host responds to the receipt of a TPDU by performing an operation and returning an acknowledgment; how should the sending host respond when the receiving host crashes before, during, or after its response?
- The sending host can employ four different strategies
 - (a) Always retransmit
 - (b) Never retransmit
 - (c) Retransmit only in S0 (Aked TPDU)
 - (d) Retransmit only in S1 (Sender didn't recive ACK)
- Unfortunately none of this strategies works in all cases!
 Consider the following events
 - A: send ACK
 - C: host crashes
 - W: data written to the application

Sliding Window Protocols

- Stop and wait ARQ protocol guarantees reliable delivery but has poor performance in terms of *filling the pipe*
- $bw \times rtt$ (bandwidth x Round trip time) represent the bit storage capacity of a path. If we don't send enough data to keep this value on the line we don't fully utilize it
- Example:
 - $bw = 1.5 \text{ Mbps}$, $rtt = 45 \text{ msec} \rightarrow bw \times rtt = 67.5 \text{ kb}$
 - Assume 1 frame = 1KB
 - Stop and wait protocol: max bit rate = 1 frame each $rtt = 1024 \times 8 / 0.045 = 182 \text{ kbps}$ (1/8 of line capacity)
- Solution: sliding window protocols

Sliding Window Protocols

- Sliding window protocols extend Stop and Wait allowing to introduce > 1 frame into the network before receiving an ACK
- Data is commonly transmitted in both directions simultaneously (ACKs may come back together with data $\rightarrow$ piggyback)
- Sender maintains a set of sequence numbers corresponding to frames it is allowed to send (within the "sending window") $\rightarrow$ see figure (a)
- Receiver maintains a set of sequence numbers corresponding to frames it is allowed to accept (within the "receiving window")

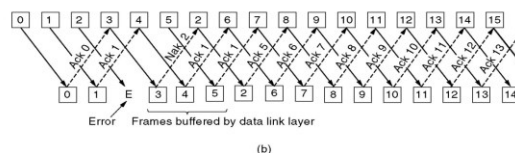
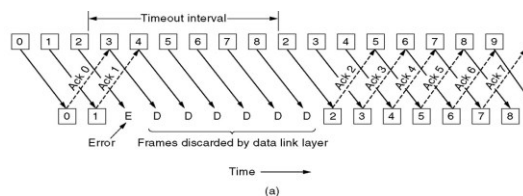


Sliding Window Protocols

- If we assume windows size = 1, the sliding window become a stop-and-wait protocol (also called 1 bit sliding window)
- Source sends a frame, then waits for corresponding ACK
- Of course this protocol “doesn’t keep the pipe full”
- When the product *delay x bandwidth* is high → the sender window is set high to operate at max efficiency
- What happens when a frame in the middle of a long sequence is damaged? Lots of frames may arrive at destination before source become aware of error
- What does the receiver do in this case?

Go back N and selective repeat

- 2 basic approach for error recovery: *go back N* and *selective repeat*



Pipelining and error recovery. Effect on an error when (a) Receiver's window size is 1 (b) Receiver's window size is large.

Sliding Window Protocols: go back N

- Go back N features:
 - Receiver:
 - It operates just like in Stop and Wait
 - Frames received after an error occurred are discarded without ACK → data link refuses to accept frames other than the next it has to deliver to network layer
 - Sender:
 - If the sender window become full before the timeout, the pipeline starts to empty
 - After timeout the sender re-sends all non ACKed frames (starting from the one damaged)

Sliding Window Protocols: selective repeat

- Selective Repeat features:
 - Receiver:
 - Erroneous frame is discarded
 - Good frames are put in buffer
 - Sender:
 - Its window has size >1
 - When the source goes in timeout only the oldest frame not ACKed is re-sent (if successful the receiver can pass all buffered frames to the network layer)
 - Selective repeat often associated to negative acknowledgement (NAK) to increase performance

Online Examples

- [Simulation](#)

User Datagram Protocol (UDP)

- UDP adds to IP multiplexing/demultiplexing and a weak error control
- UDP takes message from application layer and builds UDP segment
- No handshaking between sender/receiver entities → connectionless
- Unreliable but:
 - More subtle application level control (TCP has congestion/flow control). Real time application need minimum transmission rate and low delay (TCP doesn't fit well this model)
 - No connection set up (no delay)
 - No connection state: end terminal in TCP need to keep the state (sender/receiver buffers, seq. nr., ack, etc) → a server working on UDP can support much more active client w.r.t. TCP
 - Header is shorter (8 byte vs. 20 byte)

User Datagram Protocol (UDP)

- The use of UDP in multimedia applications is however controversial
- The lack of congestion control in UDP could result in high lost packets between source/destination and congestion in TCP sessions

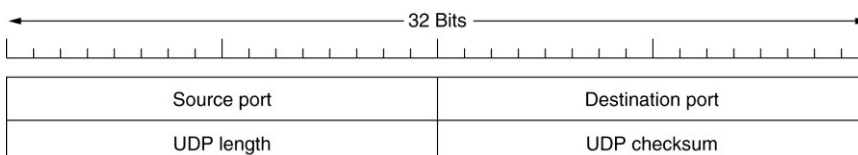
Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	typically proprietary	Typically UDP
Internet telephony	typically proprietary	Typically UDP
Network management	SNMP	Typically UDP
Routing protocol	RIP	Typically UDP
Name translation	DNS	Typically UDP

Copyright © 2005
Pearson Addison
Wesley All rights
reserved

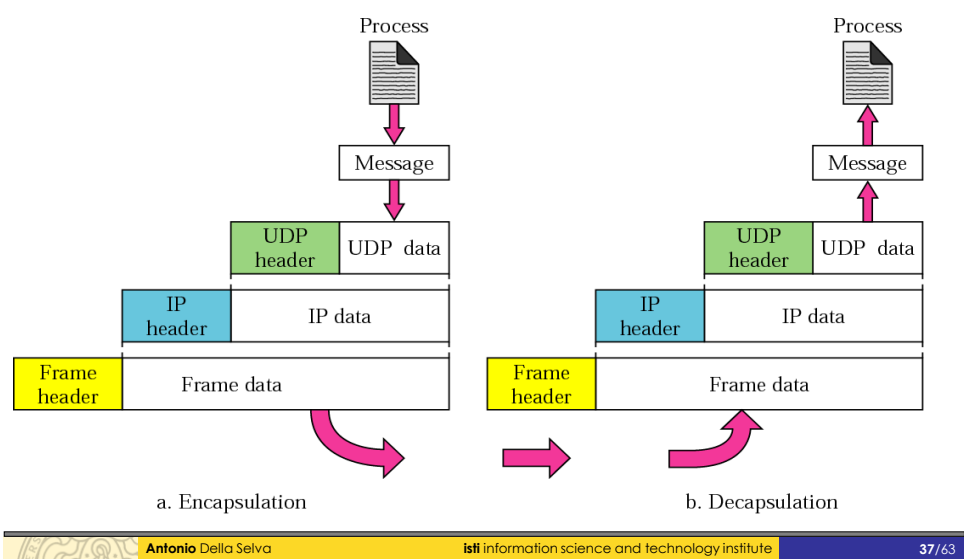
♦ Popular Internet applications and their underlying transport protocols

Introduction to UDP

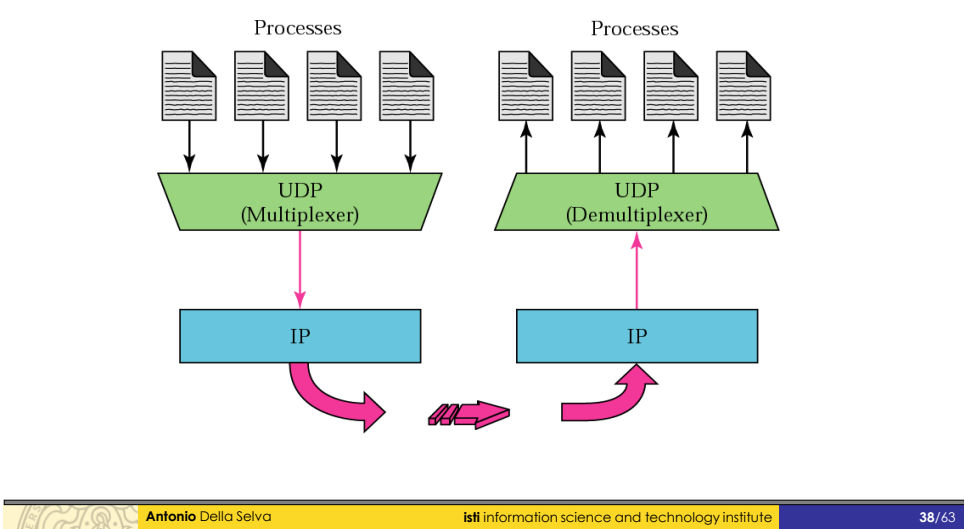
- Source port is used to answer to the original process
- UDP length includes header and data
- UDP checksum adding all 16 bit words of the segment, complementing to 1 and adding possible carry to first bit
- Why checking errors? End-to-end approach
- UDP checks but nothing is done:
 - Some implementations discard segment
 - Others may send a warning to application



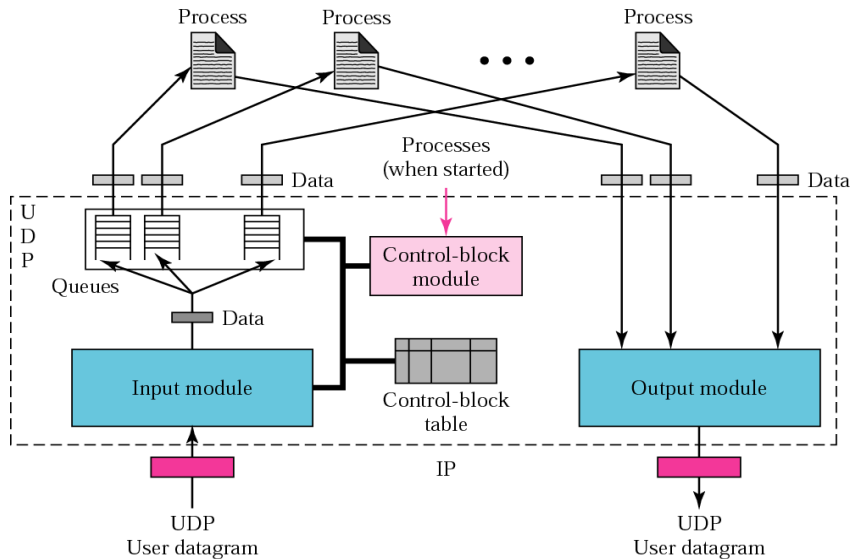
UDP: Encapsulation and decapsulation



UDP: Multiplexing and demultiplexing



UDP Design



UDP Control-Block Table

- UDP has a control-block table to keep track of the open ports.
- Each entry in this table has a minimum of four fields:
 - the state, which can be FREE or IN-USE
 - the process ID, the port number, and the corresponding queue number.

Input queues

- UDP package uses a set of input queues, one for each process.
- In this design, there is no output queues.

UDP: Control Block Module

- 1 *UDP_Control_Block_Module (process ID, port number)*
- 2 {
- 3 *Search the table for a FREE entry.*
- 4 *if (not found)*
- 5 *Delete one entry using a predefined strategy.*
- 6 *Create a new entry with the state IN-USE*
- 7 *Enter the process ID and the port number.*
- 8 *Return.*
- 9 } *// End module*

UDP: Input Module

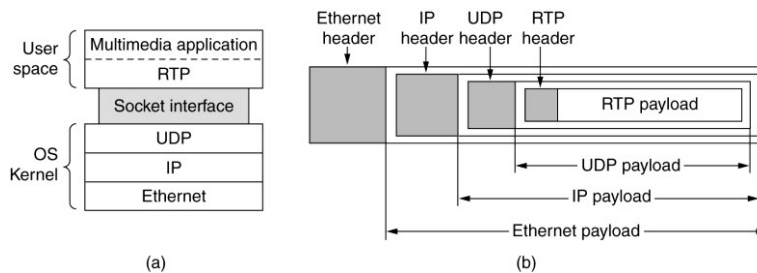
```
1 UDP_INPUT_Module (user_datagram)
2 {
3   Look for the entry in the control_block table
4   if (found)
5   {
6     Check to see if a queue is allocated
7     If (queue is not allocated)
8       allocate a queue
9   else
10    enqueue the data
11  } //end if
12 else
13 {
14   Ask ICMP to send an "unreachable port" message
15   Discard the user datagram
16 } //end else
17 Return.
18 } // end module
```

UDP: Output Module

```
•1 UDP_OUTPUT_MODULE (Data)
•2 {
•3   Create a user datagram
•4   Send the user datagram
•5   Return.
•6 }
```

The Real-Time Transport Protocol

- The increasing diffusion of multimedia real time application has determined the birth of a generic transport protocol for these applications
- RTP is usually executed over UDP
- Multimedia applications: audio, video, text flows
- Flows are passed to the RTP library that multiplexes flows and encode them into RTP packets to be placed into socket
- On the other side of the socket the kernel creates UDP packets and frame them into IP packets
- RTP can be seen as a transport protocol implemented in the application layer

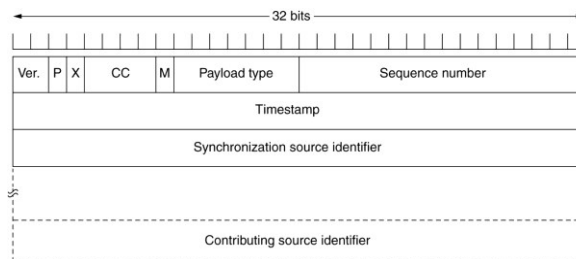


The Real-Time Transport Protocol

- Each RTP packet is progressively numbered
- Lost packet values can be interpolated
 - No re-transmission
 - No flow control, error control, ACK
- RTP payload may contain more samples encoded by the application
- A timestamp can be associated to the first sample of each packet (relative values)
- RTP header is composed of 12 bytes + some optional extensions
- RTP is usually used together with RTCP that specifies feedback, synchronization and user interface but doesn't carry any data
 - Feedback: can be used by sender to use different encoding, speed up when network works fine and slow down in case of problems (e.g. go from MP3 to PCM 8bit to delta encoding)
 - Synchronization: different flows may use different clocks, RTCP may be used to this purpose
 - Sources can also be named (e.g. ASCII text)

The Real-Time Transport Protocol

- Version: the version of the protocol (currently 2)
- P: denotes padding to obtain a multiple of 4 bytes (last byte of padding is the value of how byte have been added)
- X: denotes an extended header
- CC: number of original sources (see CS id)
- M: application specific. It can be used to mark the start of a video, the first word in an audio channel or something else
- Payload type: which encoding algorithm has been used (MP3, not compressed 8-bit audio, etc.)
- Seq. nr.: counter
- Timestamp: denotes when the first sample of packet has been created
- Synch. source id.: specifies to which flow the packet belongs to
- Contributing source identifier: optionally used to specify the list of mixed flows



RTP use scenarios

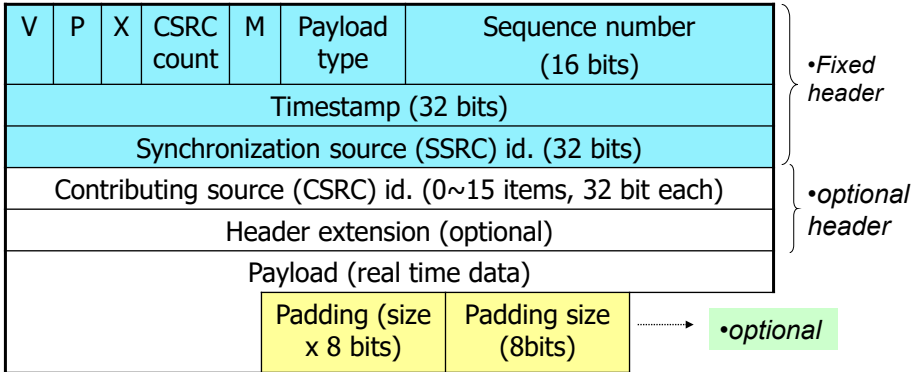
- Simple multicast audio conference
 - A multicast address and two UDP ports (for RTP and RTCP), assigned and distributed by mechanisms beyond the scope of RTP
 - Speaker send:

IP header	UDP header	RTP header	Audio data
-----------	------------	------------	------------
 - Receiver play out audio data according to RTP header
 - Sender/receiver periodically multicast report by RTCP
 - Who is participating?
 - How well is the audio quality?

RTP use scenarios – cont.

- Audio and video conference
 - Two RTP sessions, one for audio and the other for video
 - User can participate audio, video or both
 - No direct coupling at RTP level except a user use the same name in RTCP packets for both audio and video

RTP – packet format



- Version (V, 2bits): =2
- Padding(P, 1bit): If set, last byte of payload is padding size
- Extension(X, 1bit): If set, variable size header extension exists

RTP - header

- CSRC count (4 bits): number of CSRC id.
- Marker (1 bit): defined in *profile*, mark significant event
- Payload type (7 bits): Audio/Video encoding scheme
- Sequence number: random initial value, increase by one for each RTP packet; for loss detection and seq. restoration
- SSRC: identify source; chosen randomly and locally;
collision needs to be resolved
- CSRC list: id. of contributing sources, inserted by *mixer*

RTP – header - timestamp

- Reflects sampling instance of the first octet in payload
- Derived from a clock increments monotonically and linearly
- Clock frequency depends on data type; specified in *profile*
- Random initial value
- Example: CBR audio, clock increment by 1 for each sample.
If block have 160 samples, timestamp increase 160
- Consecutive RTP packets may have same timestamp:
Video
- Timestamps of consecutive RTP may not increase
monotonically: MPEG interpolated video frames

RTP timestamp vs. sequence number

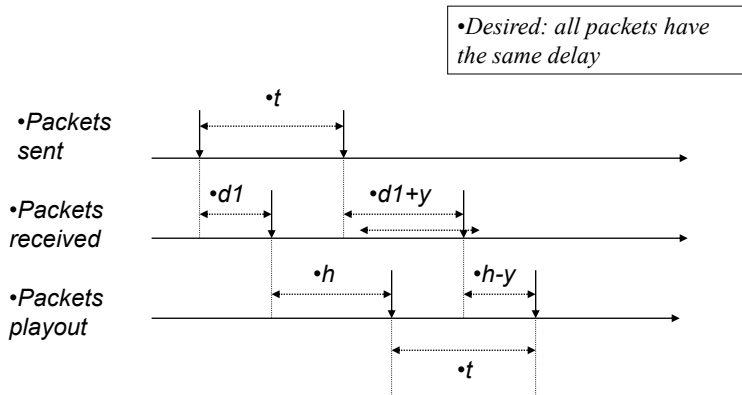
- Related, but different purposes
 - timestamp for timing reconstruction:
 - playout delay compensation (later)
 - synchronization with other sources (later)
 - sequence number for loss measurements and gap detection
- The timestamp allows receiver to do some buffering and playback samples in the wanted time instant, reducing jitter effects and allowing multiple flows synchronization (e.g. audio/video signals in digital TV)

Playout Delay

- Converts variable network delay (“jitter”) into fixed delay
 - thus, end-to-end delay is $\max(\text{jitter}) + \text{propagation delay}$
 - or, if willing to tolerate some late packets:
 - $\text{delay} < 95\% \text{ of jitter} + \text{propagation delay}$
- Propagation delay is invisible
 - and hard to measure without synchronized clocks
 - about 5 ms/1000 km one way
- Total delay should be less than 150 ms one-way
- End-to-end delay must remain constant within a talkspurt
 - otherwise gaps
- **talkspurt** is a continuous segment of speech between silent intervals

RTP – intra-media synchronization

- Reconstruction with playout buffering



RTP – cont.

- Multiplexing
 - Provided by transport address (network address + port number)
 - Teleconference with Audio and Video
 - A and V are carried in separate RTP session
 - Each session has two transport address
- Profile-specific modification
 - Marker bit
 - Header extension starts at payload section

RTCP – RTP control protocol

- Periodically transmit report to all participants;
Functions of RTCP:
 - Provide QoS feedback
 - Carry persistent id. - Canonical name (CNAME)
 - Track a user if SSRC changed (confliction, etc.)
 - Associate multiple streams from a user – A and V
 - Control the rate of RTCP packets – scalability
 - Convey minimal session control information
 - Not enough for complicated session control requirements

RTCP - types

- Sender report (SR): statistics from active sender
- Receiver report (RR): statistics from participants except active sender
- Source description item (SDS): includes CNAME
- BYE: indicates end of participation
- APP: application specific functions

- RTCP packets have a length field in header; aligned to 32 bits --- stackable
- Sent in a compound packet of at least 2 RTCP packets; example:

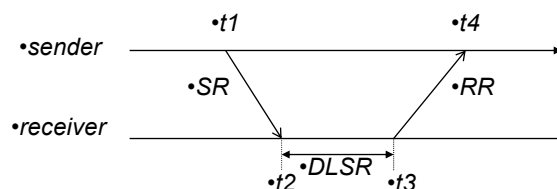


- SSRC: identify sender
- Sender info. block:
 - NTP timestamp: sent time (wallclock time or other)
 - RTP timestamp: corresponding to NTP but in formats of RTP data packet timestamp; used for intra&inter media synchronization
 - Sender's packet count & octet count
- Multiple report blocks, each block has
 - SSRC_n, fraction lost, number of lost
 - Highest sequence number received
 - Inter-arrival jitter, LSR, DLSR

RTCP – RR & SDES

- Receiver Report (RR): Similar to SR but without sender information block
- RTCP Source Description packet (SDES)
 - Containing CNAME, mandatory
 - Constant for a user
 - It may also be used to provide additional information such as the name, e-mail address, telephone number, and address of the owner or controller of the source.

RTCP – round trip time calculation



- SR packet contains: NTP ($=t_1$)
- RR packet contains: Last SR timestamp ($LSR=t_1$), Delay Since Last SR ($DLSR=t_3-t_2$)
- Roundtrip time = $t_4 - t_3 + t_2 - t_1 = t_4 - (t_3-t_2) - t_1$
 $= t_4 - DLSR - LSR$

RTCP – transmission interval

- Designed to scale from a few to thousands users
 - Problem: RTCP traffic is not self-limiting; grows linearly with number of user if sent in constant rate
 - Solution: limit control traffic to a small and known fraction of total session traffic, 5% suggested
 - Small efficiency Known: resource reservation
- Characteristics of transmission interval calc. algorithm
 - Sender occupies 25% of control traffic bandwidth
 - Calculated interval should be no less than 5 seconds
 - Trans. interval randomly varied between a range
 - Dynamic estimate average compound packet size